Servidores Web: Arquitectura, Tipos y Roles

Definición de Sistema Distribuido

❝ Cita Fundamental de Tanenbaum

"Un sistema distribuido es una colección de computadoras independientes que aparecen ante los usuarios como un único sistema coherente."

Autonomía

Cada nodo posee su propia memoria y sistema operativo.

Transparencia

El usuario final ignora la ubicación física del servidor web.

CONCEPTO CENTRAL

¿Qué es un Servidor Web?

Dimensión Hardware

Computadora física o virtual conectada a la red que almacena recursos web y responde a peticiones remotas.

Dimensión Software

Proceso o programa ejecutable que procesa peticiones HTTP/HTTPS entrantes y sirve respuestas estructuradas.

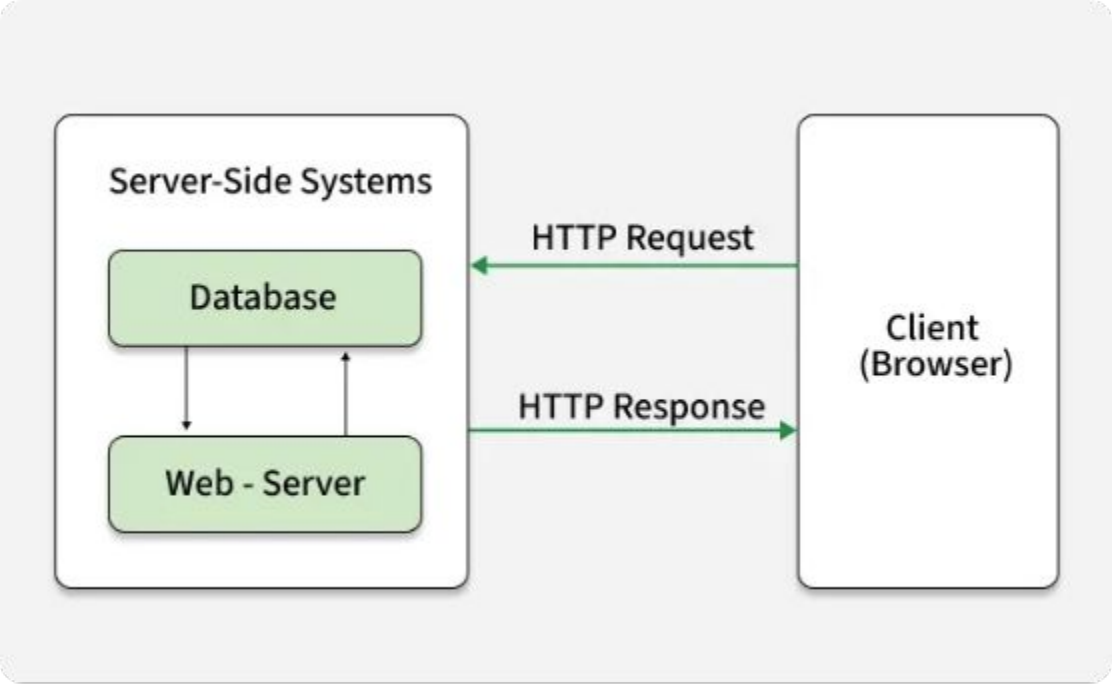
El Paradigmático Modelo Cliente-Servidor

Cliente (Navegador)

Inicia la interacción mediante el envío activo de peticiones.

Servidor (Web)

Proceso pasivo a la escucha de peticiones en puertos específicos (80/443).



MECANISMO OPERATIVO

El Ciclo Request / Response

1. Solicitud HTTP

El cliente envía un mensaje estructurado indicando recurso y método.

2. Procesamiento

El servidor valida la petición, evalúa permisos y recupera el contenido.

3. Respuesta HTTP

Se devuelve un código de estado junto con los encabezados y el cuerpo del recurso.

HTTP: El Lenguaje del Servidor Web

Protocolo Sin Estado (Stateless)

Cada petición se procesa de forma totalmente independiente a las anteriores.

Manejo de Sesión

Uso de cookies y tokens para mantener contexto sobre un protocolo stateless.

HTTPS añade una capa de cifrado TLS/SSL sobre TCP para garantizar confidencialidad e integridad.

HTTP: El Lenguaje del Servidor Web

1. ¿Qué es HTTP? ("El Lenguaje del Servidor Web")

HTTP (*Hypertext Transfer Protocol*) es el protocolo que se ubica en la **capa de aplicación** del modelo OSI/TCP-IP. Es el «idioma» estándar que usan los navegadores para pedir páginas, imágenes o datos, y que los servidores entienden para devolver una respuesta.

2. Protocolo Sin Estado (*Stateless*)

> *"Cada petición se procesa de forma totalmente independiente a las anteriores."*

- **El concepto:** HTTP es «amnésico» por diseño. El servidor web procesa una solicitud, entrega la respuesta y **olvida inmediatamente quién hizo la petición**. Si haces clic en tres enlaces de la misma página, para el servidor son tres personas completamente distintas haciendo peticiones aisladas.
- **Por qué Tanenbaum destaca esto:** Este diseño simplifica enormemente el servidor. No tiene que conservar memoria ni almacenar el estado de millones de usuarios conectados al mismo tiempo, lo que hace que el sistema sea altamente escalable y tolerante a fallos.

HTTP: El Lenguaje del Servidor Web

3. Manejo de Sesión

"Uso de cookies y tokens para mantener contexto sobre un protocolo stateless."

- **El problema:** Si HTTP no recuerda nada, ¿cómo es posible que entres a un carrito de compras, agregues productos o inicies sesión en una cuenta sin tener que poner tu contraseña en cada clic?
- **La solución:** Para «simular» memoria sobre un protocolo que no la tiene, se utilizan mecanismos de contexto:
  - **Cookies:** Pequeños fragmentos de texto que el servidor le entrega al navegador. En cada nueva petición, el navegador le reenvía esa cookie al servidor como diciendo: *"Soy yo de nuevo, este es mi número de ticket"*.
  - **Tokens (ej. JWT):** Identificadores digitales firmados que el cliente envía en la cabecera (*header*) de la petición para demostrar que está autenticado.

4. Seguridad con HTTPS

"HTTPS añade una capa de cifrado TLS/SSL sobre TCP para garantizar confidencialidad e integridad."

- **De HTTP a HTTPS:** HTTP transmite la información en texto plano. Si alguien intercepta los paquetes en la red, puede leer tus datos o modificarlos.
- **Capa TLS/SSL:** HTTPS (*HTTP Secure*) toma el mensaje HTTP y, antes de enviarlo por TCP, lo pasa por una capa de cifrado (**TLS/SSL**).
- **Las dos garantías clave:**
  1. **Confidencialidad:** Nadie en el camino puede leer el contenido (si interceptan los datos, solo verán texto cifrado incomprensible).
  2. **Integridad:** Nadie en el camino puede alterar el contenido sin que el cliente o el servidor lo detecten.

Soporte en la Capa de Transporte: TCP

Handshake de 3 Vías

Establecimiento de conexión fiable mediante SYN, SYN-ACK, ACK antes del intercambio HTTP.

Orientado a Conexión

Garantiza entrega ordenada y libre de errores de los datos HTML/multimedia.

Puertos Estándar

Puerto 80 reservado para HTTP y Puerto 443 para comunicaciones HTTPS cifradas.

Arquitectura Interna del Servidor Web

Aceptador

Escucha conexiones entrantes en la interfaz de red.

Módulo de Trabajo

Procesa la solicitud y ejecuta la lógica de negocio necesaria.

Cache / File System

Recupera archivos del disco o memoria RAM secundaria.

Multihilo vs Dirigido por Eventos

Modelo Multihilo (Thread-Per-Request)

Asigna un hilo del sistema operativo a cada cliente. Alto consumo de memoria por hilo.

Modelo Asíncrono (Event-Driven)

Un solo hilo maneja miles de conexiones mediante bucle de eventos e I/O no bloqueante.

Multihilo vs Dirigido por Eventos

A esto se le llama **Modelo de Concurrencia**, y existen dos enfoques principales para resolverlo.

1. Modelo Multihilo (*Thread-Per-Request*)

> *"Asigna un hilo del sistema operativo a cada cliente. Alto consumo de memoria por hilo."*

- **¿Cómo funciona?** Imagina un restaurante tradicional. Cada cliente que entra por la puerta recibe **un mozo exclusivo**. Ese mozo toma el pedido, va a la cocina, **se queda parado esperando** a que el cocinero prepare la comida, y cuando está lista, se la lleva al cliente.
- **En el Servidor Web (Ej. Apache HTTP Server tradicional):** Cada vez que un navegador realiza una petición, el servidor crea o asigna un **hilo (*thread*) de ejecución del sistema operativo** dedicado a atender a esa persona de principio a fin. Mientras el hilo espera que el disco duro lea un archivo o que la base de datos responda, **el hilo queda bloqueado (*Blocking I/O*) sin hacer nada**.
- **Su desventaja principal:** Crear y mantener hilos cuesta memoria y recursos de CPU (cambio de contexto o *context switching*). Si entran 10,000 usuarios simultáneos, necesitas 10,000 hilos. El servidor agotará la memoria RAM intentando administrarlos a todos y colapsará (el famoso problema **C10K**).

Multihilo vs Dirigido por Eventos

2. Modelo Asíncrono o Dirigido por Eventos (*Event-Driven*)

"Un solo hilo maneja miles de conexiones mediante bucle de eventos e I/O no bloqueante."

- **¿Cómo funciona?** Siguiendo la analogía del restaurante, imagina ahora a **un solo mozo hyper-eficiente** con una libreta. El mozo toma el pedido del cliente 1, lo entrega a la cocina e **inmediatamente** va a tomar el pedido del cliente 2 sin quedarse esperando. Cuando la cocina le avisa (un evento): *"¡La comida del cliente 1 está lista!"*, el mozo interrumpe lo que está haciendo un segundo, sirve la comida y sigue atendiendo a los demás.
- **En el Servidor Web (Ej. Nginx, Node.js):** Un único hilo ejecuta un **Bucle de Eventos (*Event Loop*)**. El servidor delega las tareas lentas (leer disco, consultar base de datos, esperar la red) al sistema operativo con llamadas **no bloqueantes (*Non-blocking I/O*)**. Cuando los datos están listos, el sistema operativo genera un "evento" y el hilo lo procesa en milisegundos.
- **Su ventaja principal:** Como no crea miles de hilos, el consumo de memoria RAM es insignificante e independiente del número de conexiones inactivas. Un solo proceso puede sostener decenas de miles de conexiones simultáneas sin despeinarse.

Criterio	Modelo Multihilo	Modelo Dirigido por Eventos
Asignación de recursos	1 Hilo/Proceso por cliente	1 Hilo atiende miles de eventos
Comportamiento I/O	Bloqueante (*Blocking*)	No bloqueante (*Non-blocking*)
Consumo de memoria	**Alto** (Escala linealmente con las conexiones)	**Muy bajo** y constante
Punto fuerte	Ideal para tareas pesadas de CPU por usuario	Ideal para alta concurrencia y servir miles de conexiones I/O

Servidores Estáticos vs Dinámicos

Servidor Estático

Retorna archivos tal como están almacenados en disco (HTML, CSS, imágenes). Sin procesamiento en tiempo real.

Servidor Dinámico

Ejecuta código de backend, consulta bases de datos y genera el HTML en tiempo de ejecución.

Servidores Estáticos vs Dinámicos

1. Servidor Estático (*Static Web Server*)

"Retorna archivos tal como están almacenados en disco (HTML, CSS, imágenes). Sin procesamiento en tiempo real."

- **¿Cómo funciona?** El servidor actúa como un **librero eficiente**. Guarda en su disco duro archivos terminados (`.html`, `.css`, `.js`, imágenes, PDFs). Cuando un cliente (navegador) solicita un recurso (por ejemplo, `[sitio.com/contacto.html](https://sitio.com/contacto.html)`), el servidor busca exactamente ese archivo en el disco y lo envía por la red sin modificarle ni una sola letra.
- **Analogía:** Imagina una librería con **revistas ya impresas**. Llegas, pides la revista de este mes, el empleado la toma del estante y te la entrega. La revista es idéntica para ti y para cualquier otra persona que la pida.
- **Características principales:**
  - **Ultra rápido:** Solo requiere lectura de disco/memoria e I/O de red.
  - **Bajo consumo de recursos:** No necesita ejecutar lógica compleja ni consultar bases de datos.
  - **Seguridad elevada:** Al no ejecutar código interno, la superficie de ataque para hackers es mínima.
  - **Tecnologías típicas:** Nginx, Apache (en modo básico), Amazon S3, GitHub Pages.

Servidores Estáticos vs Dinámicos

2. Servidor Dinámico (*Dynamic Web Server / Application Server*)

"Ejecuta código de backend, consulta bases de datos y genera el HTML en tiempo de ejecución."

- **¿Cómo funciona?** El servidor actúa como un **cocinero que prepara el plato en el momento**. Cuando llega una petición, el servidor no entrega un archivo estático del disco. En su lugar, pone a trabajar a un intérprete o motor de programación (Node.js, Python, PHP, Java), consulta una base de datos (por ejemplo, PostgreSQL o MySQL), combina los datos con una plantilla y **fabrica una página HTML totalmente personalizada** en ese instante para enviársela al cliente.
- **Analogía:** Imagina un restaurante donde pides un **plato a la carta con ingredientes a tu gusto**. El cocinero toma la orden, busca los ingredientes en la heladera (base de datos), los cocina (ejecuta código) y te entrega el plato recién hecho. Tu plato será distinto al del cliente de la mesa de al lado.
- **Características principales:**
  - **Personalizado:** Permite mostrar feeds de usuarios, carritos de compra, paneles de control o muros de redes sociales.
  - **Mayor tiempo de respuesta (Latencia):** Debe calcular y procesar antes de responder.
  - **Mayor consumo de CPU y Memoria:** Ejecutar código y gestionar conexiones a bases de datos exige más hardware.
  - **Tecnologías típicas:** Node.js (Express), Python (Django/FastAPI), PHP (Laravel), Java (Spring Boot).

Servidores Estáticos vs Dinámicos

Criterio	Servidor Estático	Servidor Dinámico
Origen del contenido	Archivos físicos preexistentes en disco.	Generado en memoria en tiempo de ejecución.
Lógica interna	Ninguna (solo transferencia de datos).	Ejecución de scripts, algoritmos y reglas de negocio.
Base de Datos	No requiere.	Sí, consulta y guarda información de forma continua.
Rendimiento / Velocidad	Extrema (milisegundos).	Variable (depende de la complejidad del código y la BD).

Rol 1: Servidor de Contenido Estático

Optimizado para operaciones de E/S de disco a red a ultra alta velocidad sin overhead de ejecución.

```
GET /images/logo.png HTTP/1.1
 Host: tanenbaum-demo.org

 HTTP/1.1 200 OK
 Content-Type: image/png
```

0.1ms

Tiempo de respuesta típico en cache

Rol 2: Servidor de Aplicaciones

Lógica de Negocio

Ejecuta scripts y frameworks de backend (Node.js, Python, Java, PHP).

Acceso a Datos

Mantiene conexiones persistentes y transaccionales con Bases de Datos.

CGI / WSGI

Usa interfaces estandarizadas para comunicar el web server con la aplicación.

Rol 3: Proxy Inverso (Reverse Proxy)

Seguridad

Oculta la existencia y direcciones IP reales de los servidores de origen internos.

Terminado TLS

Libera a los servidores internos del cómputo intensivo de descifrado SSL/TLS.

Rol 4: Balanceador de Carga (Load Balancer)

Round Robin

Distribuye las peticiones secuencialmente entre todos los nodos del cluster.

Least Connections

Redirige el tráfico al servidor con menor número de conexiones activas.

IP Hash

Mantiene la afinidad de cliente asignando peticiones según la dirección IP del cliente.

Rol 5: Servidor de Caching (Web Cache)

Almacena respuestas HTTP de forma temporal cerca del usuario para reducir la latencia de red y la carga del origen.

Principio de Localidad

Aprovecha que los mismos recursos son solicitados repetidamente por múltiples usuarios.

Reducción de Latencia

Evita traversals completos por la red de área amplia (WAN) respondiendo desde la RAM local.

Content Delivery Networks (CDNs)

Nodos Edge

Red geográfica distribuida de servidores web ubicados en los límites de la red.

Enrutamiento Anycast

Redirige automáticamente la petición del usuario al servidor más cercano latencialmente.

Software Servidor: Apache vs Nginx

Apache HTTP Server

Arquitectura modular clásica basada en procesos/hilos. Configuración flexible (.htaccess).

Nginx

Arquitectura asíncrona guiada por eventos. Diseñado para alta concurrencia y bajo consumo de RAM.

El C10K Problem y Soluciones

El Reto C10K

Incapacidad de los servidores multihilo tradicionales para manejar 10,000 conexiones concurrentes.

I/O Multiplexing

Uso de llamadas al sistema operacional como epoll (Linux) y kqueue (BSD).

PROTECCIÓN E INTEGRIDAD

Aspectos de Seguridad en el Servidor

WAF (Web App Firewall)

Filtra y monitorea el tráfico HTTP buscando patrones maliciosos (SQLi, XSS).

Mitigación DDoS

Protección ante ataques de denegación de servicio distribuidos mediante rate limiting.

Archivos de Log del Servidor Web

Access Log

Registra cada petición entrante: IP cliente, fecha, método, URL, estado HTTP y User-Agent.

Error Log

Registra fallos internos del servidor, errores de sintaxis en scripts o problemas de permisos de archivos.

Métricas de Rendimiento del Servidor

Throughput (RPS)

Número de peticiones procesadas exitosamente por segundo.

Latencia (TTFB)

Tiempo hasta recibir el primer byte de respuesta desde el servidor.

Conexiones Activas

Cantidad de sockets TCP actualmente abiertos en el servidor.

Hosting Virtual (Virtual Hosting)

Basado en IP

Múltiples sitios web en una misma máquina, cada uno con una IP dedicada.

Basado en Nombre (SNI)

Múltiples sitios comparten la misma dirección IP usando el encabezado Host de HTTP.